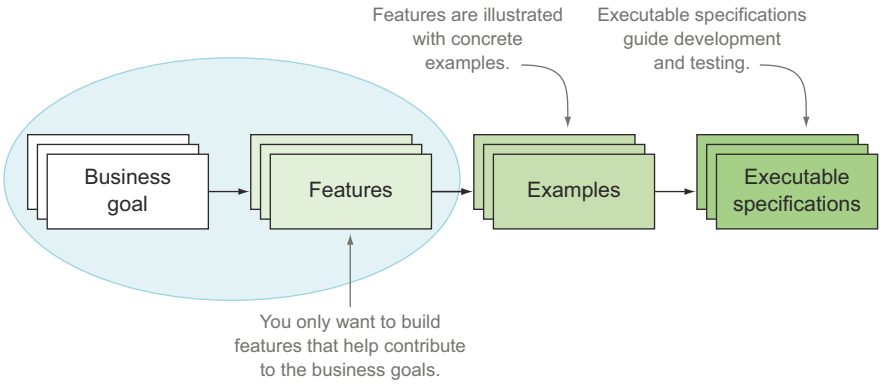


Figure 4.1 In this chapter we'll focus on how to discover and describe the features that will enable your application to achieve its business goals.

- A *user story* is a way of breaking the feature down into more manageable chunks to make them easier to implement and deliver.
- BDD practitioners use *concrete examples* to build up a *shared understanding* of how a feature should behave. These examples also help flush out and clarify *ambiguities and uncertainties* in the requirements.
- *Managing uncertainty* plays a major role in BDD practices. When they've identified areas of uncertainty, experienced BDD practitioners avoid committing to a definitive solution too early, keeping their options open until they know enough to be able to deliver the most appropriate solution for the problem at hand. This approach is known as *Real Options*.
- *Deliberate Discovery* tries to reduce project risk by managing uncertainty and ignorance proactively wherever possible.
- The examples that illustrate the features help drive the development process, becoming *automated acceptance criteria* that the developers use as a guide when they implement the features.

Let's look at these points in more detail.

4.1 What is a "feature"?

In Agile projects, developers use lots of different words to describe what they want to build (see figure 4.2). Epics, capabilities, themes, requirements, features, use cases, user stories, tasks. Confusing? Guilty as charged. The Agile community has done a relatively poor job of defining these concepts in a clear and universally understood way. Different methodologies and different teams use varying—sometimes contradictory—terms. And although the terms "user story," "epic," and "theme," for example, do actually have a long tradition in Agile and Scrum circles, many teams still waste long hours debating what terminology they should use, or whether a particular requirement should be called a story, a feature, an epic, or something else entirely.